SPEND SMART

A PROJECT REPORT

Submitted by

GOPIKA V
(241501058)

in partial fulfilment of the End Semester Examination for the course

CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF TECHNOLOGY
ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



RAJALAKSHMI ENGINEERING COLLEGE
(An Autonomous Institution, Affiliated to Anna University, Chennai)
Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "SPEND SMART" is the bonafide work of GOPIKA V (241501058), who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

Dr. M. Ayyadurai, M.E., Ph.D.,
HEAD OF THE DEPARTMENT
Department of Artificial Intelligence and Machine Learning
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Ms. Arulmozhi
COURSE FACULTY
Assistant Professor
Department of Artificial Intelligence and Machine Learning
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on ……

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, GOPIKA V (241501058), hereby declare that the project report titled “SPEND SMART”, submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: Chennai
Date: October 2026

SIGNATURE OF THE STUDENT
GOPIKA V
241501058

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman **Mr. S. Meganathan**, Chairperson **Dr. (Mrs.) Thangam Meganathan** and Vice-Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.**, for providing the infrastructure and facilities that made this work possible.

I express my sincere gratitude to our Chairman Thiru. S. Meganathan, Chairperson and Vice-Chairman for providing the infrastructure and facilities that made this work possible.

I thank **Dr. M. Ayyadurai, M.E., Ph.D.,** Head of the Department, Department of Artificial Intelligence and Machine Learning, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, **Ms. Arulmozhi**, for the valuable guidance, reviews and feedback that shaped this application and report. I also acknowledge Zoho Corporation for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

GOPIKA V

ABSTRACT

Students who live on a fixed allowance often record their spending irregularly in notebooks, phone notes or from memory. As a result they cannot easily see where their money goes, cannot compare their spending with a plan, and usually discover that they have overspent only after the month has ended. A simple, private and always available tool for recording expenses and monitoring a monthly budget is therefore useful for college students.

This project presents SPEND SMART, a multi-student expense tracking application developed on the Zoho Creator low-code platform. The application is built around two forms: Expenses, which stores the date, name, category, amount, payment method and optional description of each expense with an auto-generated Expense ID, and Monthly Budget, which stores a month, a budget amount and optional notes. Deluge scripts check that amounts are positive and compare the spending of the month with the budget; when spending reaches 80 percent of the budget or exceeds it, an e-mail alert is sent to the student through an integration with Brevo. Six reports present all expenses, today's expenses, this month's expenses, category-wise totals, payment-method totals and the monthly trend, and a dashboard shows summary cards, a category pie chart, a monthly bar chart, a recent-expenses table and a colour-coded budget status. Every student can open only his or her own records.

Seventeen test cases are defined for validation, budget comparison, alerts, reports, the dashboard and data privacy, and the report records the results that are available. The project shows that a useful personal-finance application can be configured quickly on a low-code platform, maintained without conventional programming, and extended later. Sharing of expenses within a group of students is identified as a future enhancement and is not part of this version.

Keywords: *Low-Code Development; Zoho Creator; Deluge Scripting; Expense Tracking; Budget Alerts; Brevo Integration*

PROJECT AT A GLANCE

Attribute	Details
Project Title	SPEND SMART
Application Domain	Education / Personal Finance Management
Student Name & Register No.	GOPIKA V – 241501058
Zoho Creator Edition	Education – Trial version
Application Name	SPEND SMART
Published / Live URL	https://creator.zoho.in/appbuilder/gopika.v.2024.aiml_rajala820/student-expense-tracker/page/DashBoard/edit
Demo Credentials (per role)	Administrator: testadmin@gmail.com Student: teststudent@gmail.com Faculty: testfaculty@gmail.com
Number of Forms / Subforms	2 / 0
Number of Reports (by type)	List: 5 | Kanban: 0 | Calendar: 0 | Pivot/Chart: 1 | Others: 0
Number of Pages / Dashboards	1 – SPEND SMART Dashboard
Number of Workflows	Form: 3 | Schedule: 0 | Approval: 0 | Blueprint: 0 | Batch: 0
Deluge Functions / Custom Actions	3 – DS-01, DS-02, DS-03
Integrations	Brevo (Contacts) – budget alert e-mails to students
Roles & Profiles Defined	Student (own data only); Student – Group Context (planned)
Environments Used	Single environment – application developed and published directly
Date of Production Deployment	

TABLE OF CONTENTS

ABSTRACT iv
PROJECT AT A GLANCE v
LIST OF TABLES ix
LIST OF FIGURES x
LIST OF ABBREVIATIONS xi
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 2
1.6 Organisation of the Report 2
LITERATURE SURVEY AND SYSTEM STUDY 3
2.1 Study of the Existing System 3
2.2 Limitations of the Existing System 3
2.3 Low-Code versus Traditional Development 4
2.4 Literature Survey 5
2.5 Emerging Trends in Low-Code Development 6
2.6 Proposed System 6
REQUIREMENT ANALYSIS 7
3.1 Stakeholders and User Roles 7
3.2 Functional Requirements 7
3.3 Non-Functional Requirements 9
3.4 Hardware and Software Requirements 9
3.5 Use Case Model 10
SYSTEM DESIGN 12
4.1 System Architecture 12
4.2 Data Model 13
4.3 Form Design 14
4.4 Process and Workflow Design 15
4.5 User Interface and Navigation Design 16
4.6 Access Control Design 17
IMPLEMENTATION 18
5.1 Application Setup 18
5.2 Forms and Data Capture 18

5.3 Reports 19
5.4 Pages and Dashboards 20
5.5 Deluge Scripting 21
5.6 Workflows and Automation 23
5.7 Integrations 24
5.8 User Management and Role-Based Access 25
5.9 AI and Analytics Features 25
5.10 Pro-Code Extension 25
5.11 Mobile and Portal Access 26
TESTING AND DEBUGGING 27
6.1 Testing Strategy 27
6.2 Test Cases 27
6.3 Debugging Deluge Scripts 30
6.4 Defect Log 31
6.5 User Acceptance Testing 31
DEPLOYMENT, SECURITY AND GOVERNANCE 32
7.1 Environments and Deployment Strategy 32
7.2 Performance Optimisation 32
7.3 Security Features 33
7.4 Governance and Administration 34
RESULTS AND DISCUSSION 35
8.1 Application Output 35
8.2 Achievement of Objectives 36
8.3 Comparison with the Existing System 37
8.4 Limitations of the Low-Code Approach 37
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 38
CONCLUSION AND FUTURE ENHANCEMENTS 39
9.1 Conclusion 39
9.2 Future Enhancements 39
9.3 Learning Outcomes and Career Relevance 39
REFERENCES 40
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 41
APPENDIX B – COURSE OUTCOME MAPPING 44
APPENDIX C – COMPLETE DELUGE SOURCE CODE 45
APPENDIX D – USER MANUAL 47
D.1 Accessing the Application 47
D.2 Instructions for Recording Expenses and Budgets 47

D.3 Instructions for Viewing Reports and the Dashboard 47
APPENDIX E – EVALUATION RUBRIC 48
APPENDIX F – VIVA-VOCE RECORD SHEET 50

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application....4
Table 2.2: Summary of Literature Survey....5
Table 3.1: Stakeholders and User Roles....7
Table 3.2: Functional Requirements....7
Table 3.3: Non-Functional Requirements....9
Table 3.4: Hardware and Software Requirements....9
Table 3.5: Use Case Description – Record Expense....10
Table 4.1: Entities and Relationships....13
Table 4.2: Field Specification – Expenses Form....14
Table 4.3: Field Specification – Monthly Budget Form....14
Table 4.4: Role–Permission Matrix....17
Table 5.1: Reports Implemented....19
Table 5.2: Deluge Scripts Implemented....21
Table 5.3: Workflows Implemented....23
Table 5.4: Integrations Implemented....24
Table 6.1: Test Cases....27
Table 6.2: Defect Log....31
Table 6.3: User Acceptance Feedback....31
Table 7.1: Deployment Checklist....32
Table 7.2: Performance Optimisation Results....33
Table 7.3: Security Controls Implemented....33
Table 8.1: Mapping of Objectives to Outcomes....36
Table 8.2: Existing System versus Proposed Application....37
Table A.1: Feature Coverage Checklist....41
Table B.1: Mapping of Course Outcomes to Report Chapters....44
Table E.1: Evaluation Rubric for the Individual Project....48
Table E.2: Performance Level Descriptors....49
Table F.1: Viva-Voce Questions and Responses....50
Table F.2: Marks Summary....50

LIST OF FIGURES

Figure 2.1: As-Is Process Flow of Manual Expense Tracking 3
Figure 3.1: Use Case Diagram of SPEND SMART 10
Figure 4.1: System Architecture of SPEND SMART 12
Figure 4.2: Entity–Relationship Diagram of SPEND SMART 13
Figure 4.3: Workflow / Process Flow for Expense Entry and Budget Monitoring 15
Figure 4.4: Navigation Structure of SPEND SMART 16
Figure 5.1: SPEND SMART – Application Home and Navigation Sections 18
Figure 5.2: Expenses Form in the Form Builder 18
Figure 5.3: Monthly Budget Form in the Form Builder 19
Figure 5.4: All Expenses Report 20
Figure 5.5: SPEND SMART Dashboard Page 21
Figure 5.6: Form Workflow Configuration 24
Figure 5.7: Brevo Connection and Integration Configuration ... **Error! Bookmark not defined.**
Figure 5.8: Roles and Permission Profiles Configuration 25
Figure 5.9: SPEND SMART on a Mobile Device 26
Figure 7.1: Environments / Publish to Production 32
Figure 8.1: Scenario Step 1 – Student sets a budget and records an expense 35
Figure 8.2: Scenario Step 2 – Dashboard updates and the budget alert is sent 35
Figure 8.3: Scenario Step 3 – Student reviews the analysis reports 36

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
Deluge	Data Enriched Language for the Universal Grid Environment
ER	Entity–Relationship
FR	Functional Requirement
LCAP	Low-Code Application Platform
NFR	Non-Functional Requirement
RBAC	Role-Based Access Control
UAT	User Acceptance Testing
UC	Use Case
UI	User Interface
URL	Uniform Resource Locator
Zia	Zoho's AI assistant

CHAPTER 1
INTRODUCTION

1.1 Background

SPEND SMART is an expense tracking application for college students. A student receives money from a limited source, such as a monthly allowance, and spends it every day on food, travel, study material, entertainment and other needs. Keeping control of this spending requires three simple habits: recording each expense when it happens, deciding in advance how much may be spent in a month, and checking regularly how the spending compares with that limit.

1.2 Problem Statement

Students who track their expenses manually have no quick way to see the total spent today or this month, to see the split of spending by category or payment method, or to compare spending with a budget. Alerts about overspending come too late, if at all, because the comparison is done by hand and only when the student remembers to do it. A private, structured and automated expense tracker is therefore required, one that records expenses with validation, keeps monthly budgets, summarises the data visually and warns the student in time. The specific problems are:

- Expenses are recorded inconsistently, with missing dates, amounts or categories, so totals cannot be trusted.
- There is no ready view of today's spending, this month's spending or the monthly trend.
- Spending cannot be grouped easily by category or payment method to recognise habits.
- Students are not warned when spending approaches or exceeds the monthly budget.
- Personal financial data must stay private to each student even though the application serves many students.

1.3 Objectives

- To design forms and fields for recording expenses and monthly budgets with validation and an auto-generated Expense ID.
- To implement positive-amount validation and the budget comparison logic using Deluge scripting and form workflows.
- To provide list and chart reports and a dashboard that show spending by day, month, category and payment method.
- To send e-mail alerts through Brevo when spending reaches 80 percent of the budget or exceeds it.

- To keep each student's data private through role permissions and record-level criteria.
- To evaluate the application through test cases and, where available, user feedback.

1.4 Scope of the Project

In scope. The application contains two forms: Expenses and Monthly Budget. It supports recording an expense with date, name, category, amount, payment method and an optional description; automatic generation of the Expense ID; mandatory-field and positive-amount validation; amounts displayed in Indian Rupees (₹); editing and deleting of expenses; setting a monthly budget with optional notes; six reports (All Expenses, Today's, This Month's, Category-wise, Payment Method and Monthly Trend) with search and filter; a dashboard with summary cards, a category pie chart, a monthly bar chart, a recent-expenses table and budget status indicators; and e-mail alerts through Brevo at 80 percent of the budget and when the budget is exceeded. The only user is the Student, and each student sees only his or her own data.

1.5 Justification for a Low-Code Approach

Zoho Creator was selected because an expense tracker consists of structured records – expenses and budgets – that are naturally represented as forms with typed fields, and because its reports, charts and dashboard pages cover the display needs without separate front-end development. Low-code platforms let developers concentrate on the business requirement instead of infrastructure and repetitive code [3], [4], [5]. Zoho Creator also provides hosting, user management, role-based access, a mobile-friendly interface, and the Deluge language for logic that cannot be configured visually [1], [2].

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance optimisation, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

In the existing way of working, a student who wants to keep track of money uses a notebook, a note on the phone or a spreadsheet, or simply remembers the major expenses. Each expense is written down, if at all, after the event, often with the category and payment method missing. At the end of the week or month the student adds up the entries by hand and compares the total with an approximate idea of how much could be spent.

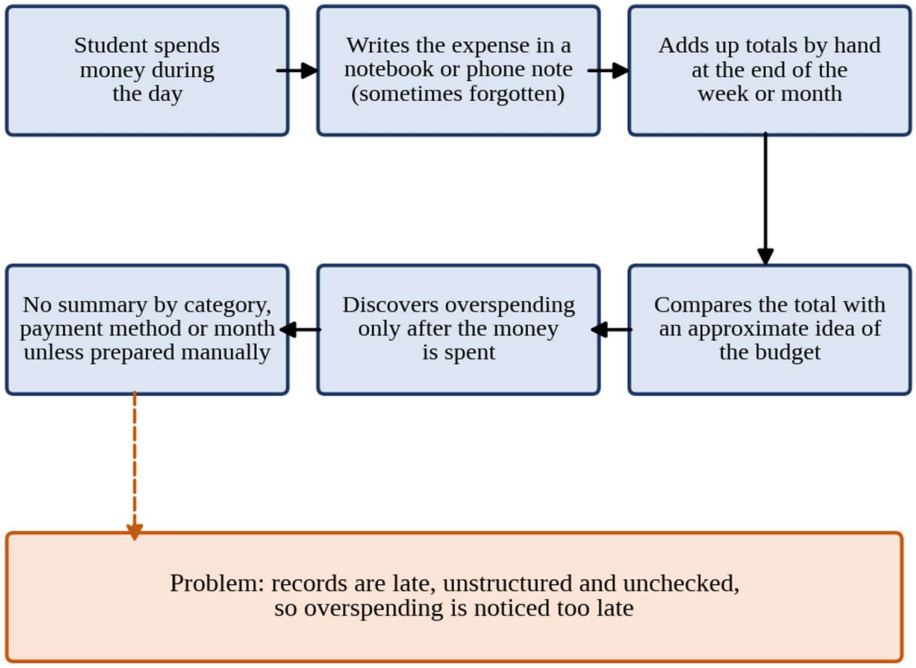


Figure 2.1: As-Is Process Flow of Manual Expense Tracking

2.2 Limitations of the Existing System

- Expenses are recorded late or not at all, so the records are incomplete.
- Entries have no fixed structure, so dates, categories and payment methods are inconsistent.
- Totals for today, the month and each category must be calculated by hand.
- There is no comparison with a budget until the student does it manually, and no warning before the budget is exceeded.
- Trends across months cannot be seen without extra effort.
- Notebooks and shared files give no controlled, private access to the data.

2.3 Low-Code versus Traditional Development

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development effort	Needs separate design and coding of the database, back end, user interface and charts.	Forms, reports, charts and the dashboard are configured visually; Deluge is used only for validation and alert logic.
Skills required	Front-end, back-end, database and deployment skills.	Data modelling, platform knowledge and basic Deluge scripting.
Hosting & infrastructure	Server, database, domain and deployment pipeline must be arranged and maintained.	Hosting, backup and scaling are provided by the Zoho Creator platform, subject to the plan.
Mobile access	A separate responsive front end or native app is required.	The application can be opened in a mobile browser or in the Zoho Creator mobile app.
Security & access control	Authentication, authorisation and per-user data separation must be coded.	Users, roles, permission profiles and record-level criteria are configured, not coded.
Maintenance & change	A change to a field or report can touch several layers of code.	A new field, report or chart can usually be changed by the owner in the builder.
Customisation limits	Almost unlimited design freedom.	Limited by the platform's field types, page builder and Deluge capabilities (see Section 8.4).
Cost	Developer time, hosting and maintenance costs.	Platform subscription according to the plan; the Education edition may be used for learning.

2.4 Literature Survey

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	Zoho Corporation [1]	Zoho Creator Help Documentation (online)	Describes forms, field types, reports, pages, charts, workflows, user management and publishing in Zoho Creator.	Reference for designing the two forms, six reports, dashboard and permissions.
2	Zoho Corporation [2]	Deluge Script Reference Guide (online)	Describes the Deluge language, form events, fetch statements, aggregate functions and the invokeurl task.	Reference for scripts DS-01 to DS-03.
3	R. Sanchis, Ó. García-Perales, F. Fraile and R. Poler, 2020 [3]	Low-Code as Enabler of Digital Transformation in Manufacturing Industry	Argues that low-code development platforms facilitate and automate the development of software applications that support enterprise needs, and benchmarks existing platforms.	Academic support for using a low-code platform to digitise a process.
4	A. C. Bock and U. Frank, 2021 [4]	Low-Code Platform	Characterises low-code platforms, citizen development and the implications for application development.	Defines the low-code platform concept used in Chapter 1.
5	D. Di Ruscio, D. Kolovos, J. de Lara, A. Pierantonio, M. Tisi and M. Wimmer, 2022 [5]	Low-code development and model-driven engineering: Two sides of the same coin?	Examines the relationship between low-code development and model-driven engineering.	Explains why form and data models are the foundation of a low-code application.

6	A. Lusardi and O. S. Mitchell, 2014 [6]	The Economic Importance of Financial Literacy: Theory and Evidence	Reviews evidence on financial literacy and its relation to planning, saving and other financial decisions.	Motivates a tool that helps students plan and monitor spending.
7	R. H. Thaler, 1999 [7]	Mental Accounting Matters	Explains how people categorise and evaluate money in mental accounts, including the role of budgets.	Supports category-wise reports and a monthly budget as aids to self-control.
8	Brevo [8]	Brevo API Documentation (online)	Describes the Brevo API, including contacts and transactional e-mail endpoints.	Reference for the budget alert integration.

2.5 Emerging Trends in Low-Code Development

Several trends in low-code development influenced the design of SPEND SMART. The first is citizen development: platforms allow people who are not professional programmers to build and maintain business applications [3], [4]. SPEND SMART therefore uses visual configuration wherever possible and keeps its Deluge scripts short. Model-driven thinking, in which forms and fields describe the data model, also links low-code to classical software engineering [5].

2.6 Proposed System

The proposed SPEND SMART application is a centralised, multi-student expense tracker developed with Zoho Creator. It stores expenses and monthly budgets in two forms, summarises them in six reports and a dashboard, and warns each student by e-mail when spending reaches 80 percent of the monthly budget or exceeds it. Each student has private access to his or her own records.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Student	Primary user of the application; each student has a separate, private account.	Records daily expenses, edits or deletes entries, sets monthly budgets, views reports and the dashboard, and receives e-mail alerts.
Student (Group Context)	Optional future user; not implemented in this version.	Would view shared expenses and shared budgets in a group; identified as a future enhancement.

3.2 Functional Requirements

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall generate a unique Expense ID automatically for every expense.	Student	High
FR-02	The system shall require the expense date, name, category and amount before an expense is saved.	Student	High
FR-03	The system shall provide drop-down fields for category and payment method.	Student	Medium
FR-04	The system shall display all amounts in Indian Rupees (₹).	Student	Medium
FR-05	The system shall allow a student to edit and delete his or her expenses.	Student	High
FR-06	The system shall allow a student to set a monthly budget for a month with optional notes.	Student	High
FR-07	The system shall accept only positive amounts for expenses and budgets.	Student	High
FR-08	The system shall organise budgets by month and compare the budget with the expenses of that month.	Student	High

FR ID	Requirement	Role(s)	Priority
FR-09	The system shall provide All Expenses (sorted by date), Today's and This Month's reports.	Student	High
FR-10	The system shall provide Category-wise and Payment Method reports.	Student	Medium
FR-11	The system shall provide a Monthly Trend report.	Student	Medium
FR-12	The system should support searching and filtering of expenses.	Student	Medium
FR-13	The system shall provide a dashboard with summary cards, a category pie chart, a monthly bar chart, a recent-expenses table and budget status indicators.	Student	High
FR-14	The system shall send an e-mail alert when spending reaches 80 percent of the monthly budget or exceeds it.	Student	High
FR-15	The system shall allow each student to access only his or her own data.	Student	High

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Usability	The expense form shall be simple enough to complete in a few seconds, and the dashboard shall show the main figures at a glance.
NFR-02	Performance	Forms, reports and the dashboard shall load within an acceptable time; reports shall use criteria so that only the student's relevant records are loaded.
NFR-03	Security and privacy	A student shall see and change only his or her own records; access shall be enforced by permissions and record-level criteria.
NFR-04	Maintainability	Categories, payment methods, reports and charts shall be changeable through configuration without reprogramming.
NFR-05	Accessibility on devices	The application shall be usable in supported browsers and on mobile devices through the browser or the Zoho Creator mobile app.
NFR-06	Data integrity	Mandatory fields and positive amounts shall be validated before a record is accepted.
NFR-07	Localisation	Amounts shall be shown in Indian Rupees (₹) for local relevance.
NFR-08	Auditability	Important changes shall be traceable through the audit trail and change history available on the platform.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator
Scripting language	Deluge (Data Enriched Language for the Universal Grid Environment)
Browser	Google Chrome / Microsoft Edge / Mozilla Firefox (latest version)
API testing tool	Not used
Other Zoho services	Brevo (contacts and e-mail) for budget alerts

Category	Specification
Mobile	Zoho Creator mobile app (Android / iOS) or a mobile browser
Client hardware	Intel Core i3/i5 or above, 8 GB RAM, internet connection

3.5 Use Case Model

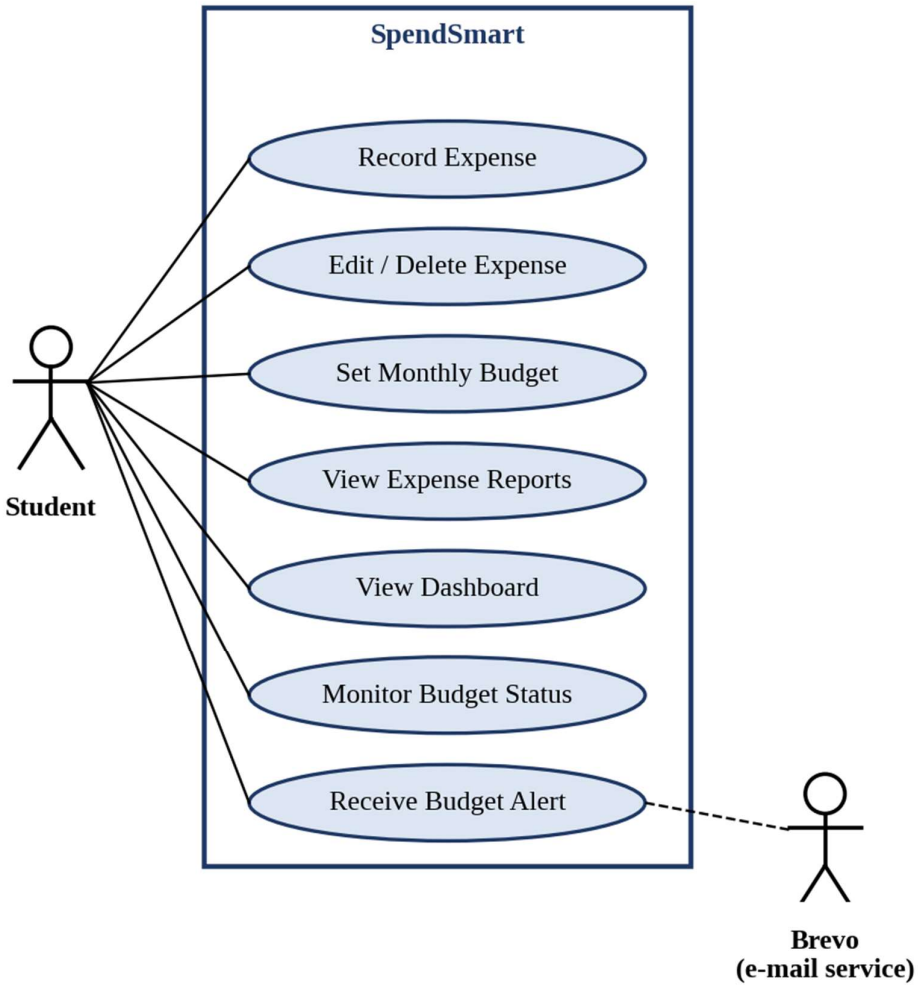


Figure 3.1: Use Case Diagram of SPEND SMART

Table 3.5: Use Case Description – Record Expense

Use Case ID	UC-01
Use Case Name	Record Expense
Actor(s)	Student
Pre-condition	The student is signed in to the application.

Main Flow	1. The student opens the Add Expense form. 2. The student enters the expense date, name, category, amount, payment method and an optional description. 3. The student submits the form. 4. The application validates the required fields and that the amount is greater than zero. 5. The application generates the Expense ID, saves the record and shows it in the reports and on the dashboard.
Alternate Flow	A required field is empty or the amount is zero or negative; the application shows a message and does not save the record.
Post-condition	The expense is stored and included in all totals and charts.
Related FR	FR-01 to FR-05, FR-07

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

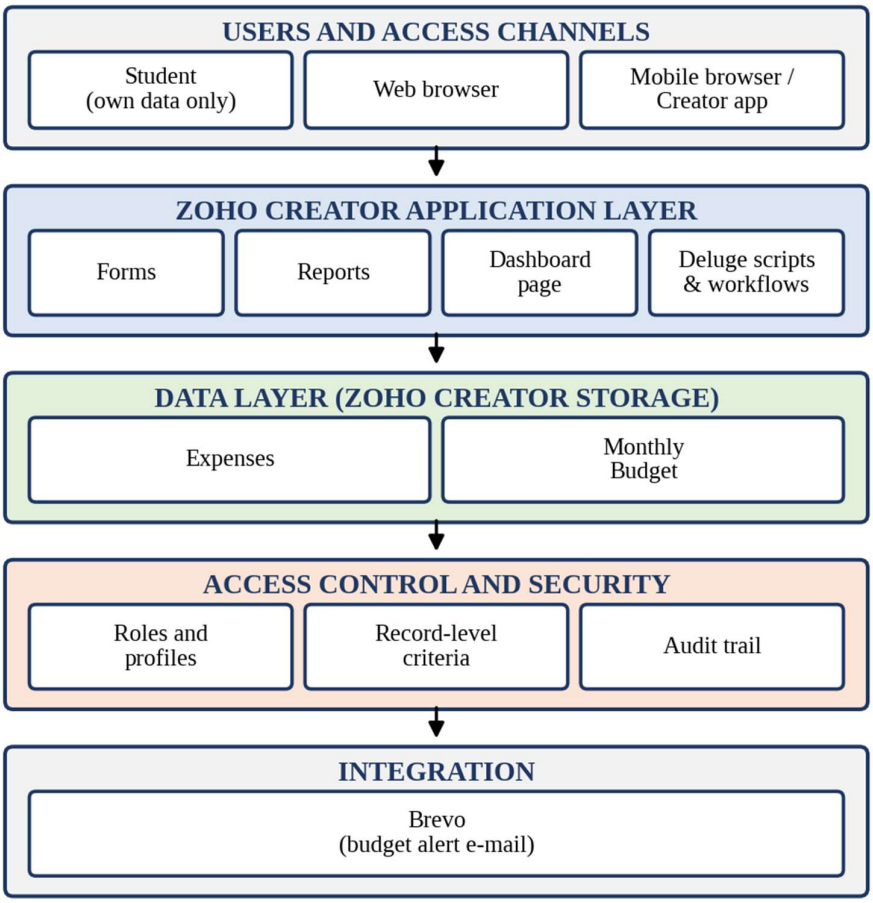
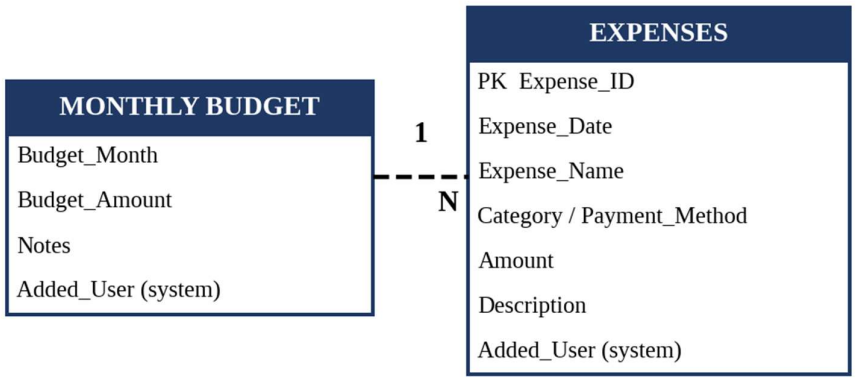


Figure 4.1: System Architecture of SPEND SMART

SPEND SMART follows a layered architecture on Zoho Creator, shown in Figure 4.1. The top layer contains the students and their access channels, a web browser or a mobile browser. The application layer contains the components configured in Zoho Creator: the Expenses and Monthly Budget forms, the six reports, the dashboard page built with the UI Builder, and the Deluge scripts and form workflows for validation and alerts.

4.2 Data Model



Logical link by month (date criteria); no lookup field is configured.

Figure 4.2: Entity–Relationship Diagram of SPEND SMART

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Expenses	Expense_ID, Expense_Date, Expense_Name, Category, Amount, Payment_Method, Description	Monthly Budget	Many-to-One (logical, by month)	Matched by month in Deluge criteria; no lookup field
Monthly Budget	Budget_Month, Budget_Amount, Notes	Expenses	One-to-Many (logical, by month)	Matched by month in Deluge criteria; no lookup field

Expenses holds one record per expense, and Monthly Budget holds one record per student per month. The budget and the expenses are not joined by a lookup field, because the relationship is by time period: all expenses dated within the budget month belong to that budget, which the scripts and the dashboard calculate with date criteria. No subforms are used because the data has no parent–child line items. Both forms also keep the system field Added User, which records the owner and supports the privacy criteria.

4.3 Form Design

Table 4.2: Field Specification – Expenses Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Expense ID	Expense_ID	Auto Number	No	Prefix EXP-	System generated
Expense Date	Expense_Date	Date	Yes	Mandatory	Date of the expense
Expense Name	Expense_Name	Single Line	Yes	Mandatory	What was bought or paid
Category	Category	Drop Down	Yes	Predefined choices	Standard categories for reports
Amount	Amount	Currency (Decimal)	Yes	Greater than 0 (DS-01); shown in ₹	Amount spent
Payment Method	Payment_Method	Drop Down	No	Predefined choices	Used by the payment-method report
Description	Description	Multi Line	No	None	Optional note
Added User	Added_User	System field	No	Set by the platform	Used for privacy criteria

Table 4.3: Field Specification – Monthly Budget Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Budget Month	Budget_Month	Date	Yes	Mandatory	Month of the budget
Budget Amount	Budget_Amount	Currency (Decimal)	Yes	Greater than 0 (DS-02); shown in ₹	Monthly spending limit

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Notes	Notes	Multi Line	No	None	Optional context or reminder
Added User	Added_User	System field	No	Set by the platform	Used for privacy criteria

Six distinct Zoho Creator field types are used across the two forms: Auto Number, Date, Single Line, Drop Down, Currency (Decimal) and Multi Line. Link names follow the platform convention of replacing spaces with underscores. Screenshots of the forms are given in Chapter 5.

4.4 Process and Workflow Design

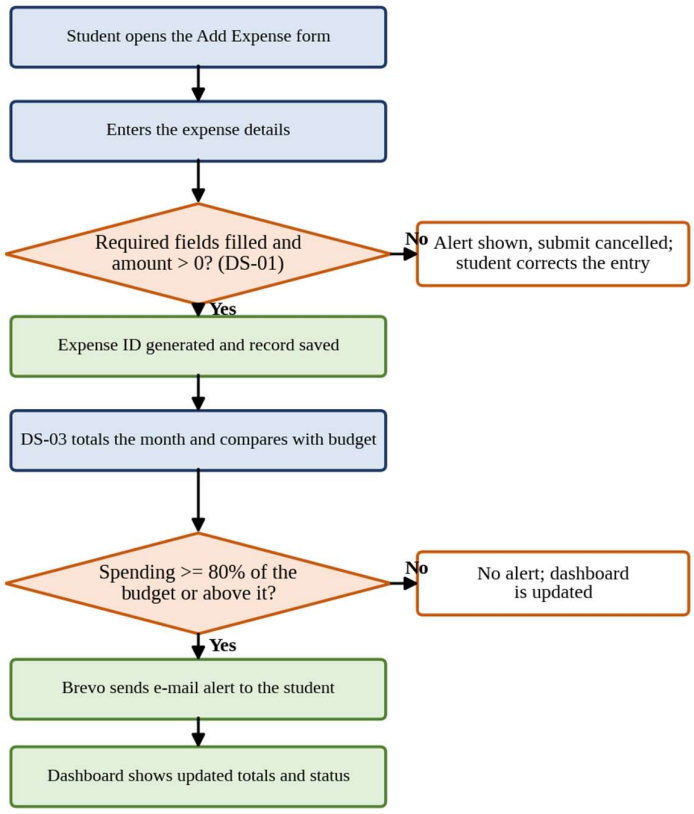


Figure 4.3: Workflow / Process Flow for Expense Entry and Budget Monitoring

Figure 4.3 shows the process. The student enters an expense on the Add Expense form. On submission the validation script DS-01 checks that the amount is greater than zero

and, together with the mandatory-field rules, rejects an incomplete record. A valid record receives its Expense ID and is saved. The script DS-03 then calculates the spending of the month, compares it with the budget of that month, and sends an e-mail alert through Brevo if the spending is at least 80 percent of the budget or exceeds it. The dashboard is then refreshed with the new totals and status.

4.5 User Interface and Navigation Design

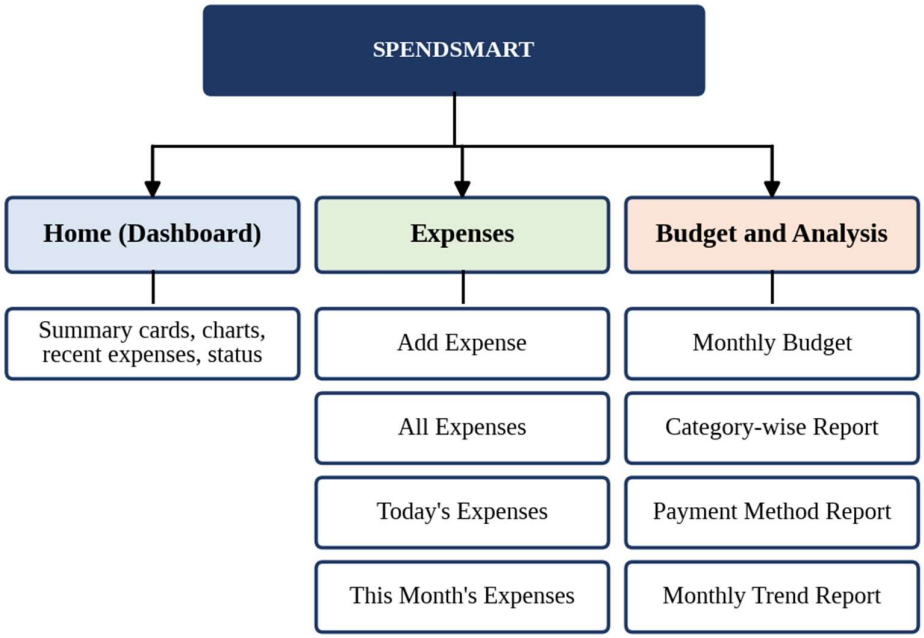


Figure 4.4: Navigation Structure of SPEND SMART

The application has a Dashboard as the home page, followed by menu groups for Expenses (Add Expense, All Expenses, Today's Expenses and This Month's Expenses), Budget (the Monthly Budget form) and Analysis (Category-wise, Payment Method and Monthly Trend reports). Every student sees the same menu but only his or her own records.

4.6 Access Control Design

Table 4.4: Role–Permission Matrix

Component	Student	Student (Group Context) – planned
Form: Expenses	A E V D (own)	Planned
Form: Monthly Budget	A E V D (own)	Planned
Report: All Expenses and period reports	V (own)	Planned
Report: Category-wise, Payment Method, Monthly Trend	V (own)	Planned
Page: Dashboard	V (own)	Planned
Shared expenses and budgets	No access	Planned

A = Add, E = Edit, V = View, D = Delete. A student can add, edit and delete only personal records; record-level criteria based on the Added User field hide the records of other students. The Group Context role is an optional future feature and has no configured access in this version.

CHAPTER 5
IMPLEMENTATION

5.1 Application Setup

The application was created in Zoho Creator from scratch, without a ready-made template, in the workspace of the student's Zoho account, and was named SPEND SMART. The first step was to identify the two entities of the problem, expenses and monthly budgets, and to create one form for each of them as designed in Chapter 4. Zoho Creator generates a default list report for every form, and these reports were refined into the six reports described in Section 5.3.

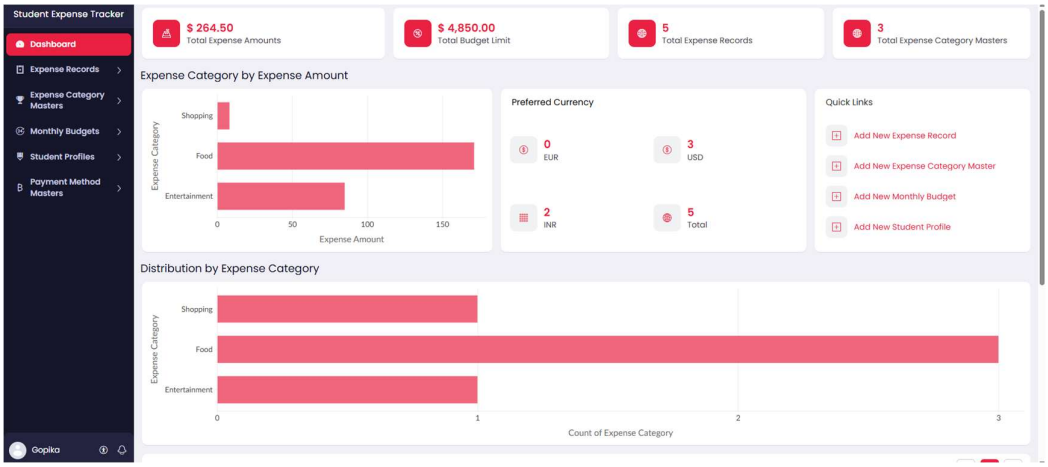


Figure 5.1: SPEND SMART – Application Home and Navigation Sections

5.2 Forms and Data Capture

The two forms were built in the Zoho Creator form builder with the field types listed in Tables 4.2 and 4.3. Mandatory fields were marked as required, drop-down fields were given predefined choices for the category and payment method, the amount fields were formatted as currency with the Indian Rupee symbol (₹), and each form received a clear success message. The Expense ID is an Auto Number field, so every expense is identified uniquely without any entry by the student.

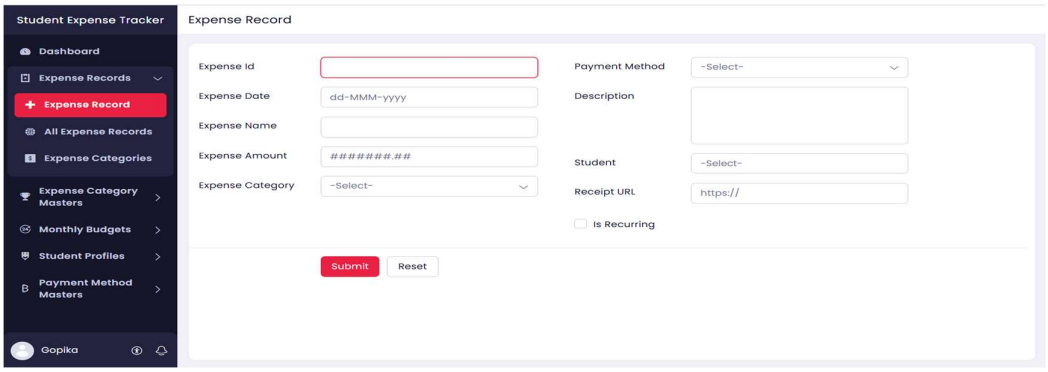


Figure 5.2: Expenses Form in the Form Builder

Monthly Budget form. This form stores the budget month, the budget amount and optional notes. The budget amount must be greater than zero, which is checked by the script DS-02. Because budgets are organised by month, a student can set a different limit for each month and can update it later. The dashboard and the alert script find the budget of a month by its date range.

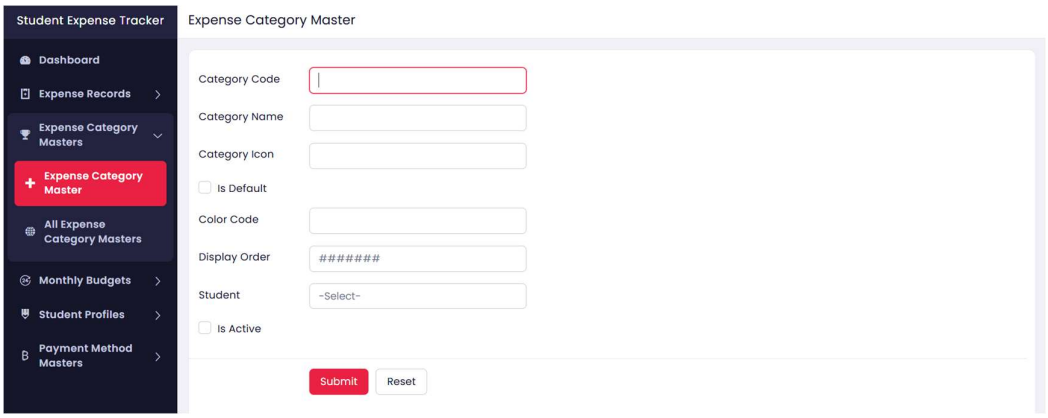


Figure 5.3: Monthly Budget Form in the Form Builder

5.3 Reports

Table 5.1: Reports Implemented

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
All Expenses	List	Expenses	Sorted by Expense Date; search and filter	Edit / Delete own records	Student
Today's Expenses	List	Expenses	Criteria: Expense Date = today	View	Student
This Month's Expenses	List	Expenses	Criteria: Expense Date in the current month	View	Student
Category-wise Expenses	List (grouped)	Expenses	Grouped by Category with totals	View	Student
Payment Method Expenses	List (grouped)	Expenses	Grouped by Payment Method with totals	View	Student

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
Monthly Trend	Chart	Expenses	Total Amount by month	View	Student

Six reports were created. Every report applies the record-level criterion that the Added User is the signed-in student, so each student sees personal data only. Quick search and column filters let a student find a particular expense, which satisfies FR-12. The Category-wise and Payment Method reports group the records and show totals so that spending habits can be recognised, and the Monthly Trend report shows how spending changes from month to month. Kanban and calendar reports were not required.

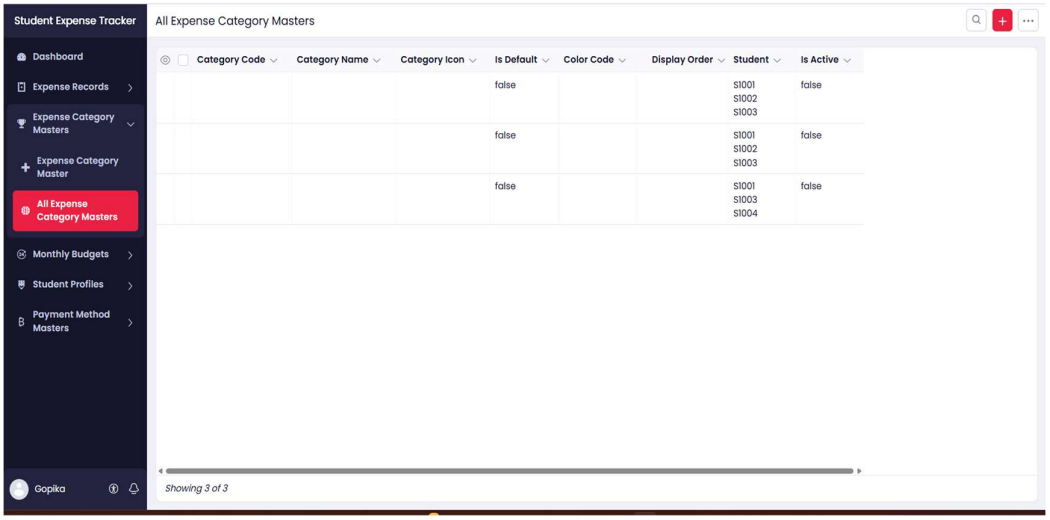


Figure 5.4: All Expenses Report

5.4 Pages and Dashboards

A dashboard page was built with the Zoho Creator UI Builder and is the home page of the application. It shows summary cards for total expenses, today's spending, month-to-date spending, the monthly budget and the remaining budget; a pie chart of spending by category; a bar chart of spending by month; and a table of recent expenses. Budget status indicators give colour-coded feedback that shows whether the student is under, at or over budget.

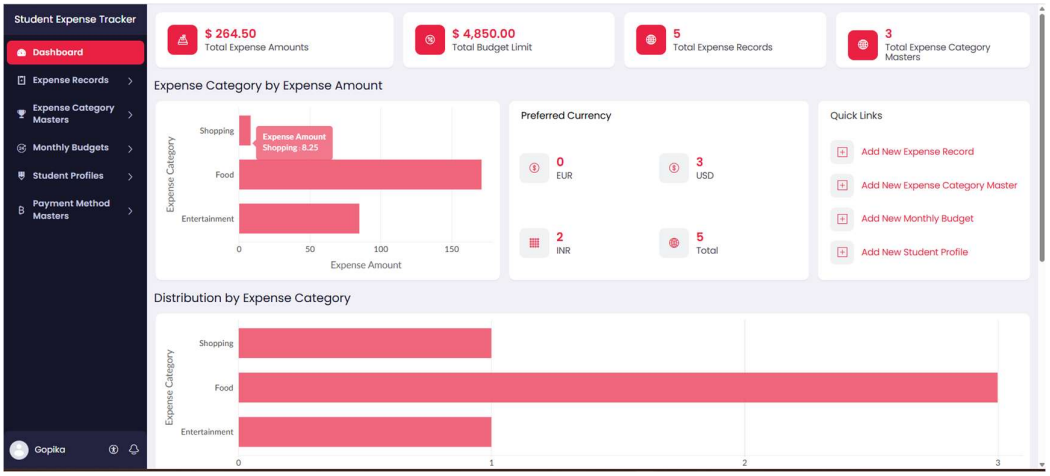


Figure 5.5: SPEND SMART Dashboard Page

5.5 Deluge Scripting

Table 5.2: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Expenses → On Add / On Edit → On Validate	Reject an expense whose amount is not greater than zero	if-else, alert, cancel submit
DS-02	Monthly Budget → On Add / On Edit → On Validate	Reject a budget whose amount is not greater than zero	if-else, alert, cancel submit
DS-03	Expenses → On Add / On Edit → On Success	Compare the month's spending with the budget and send an alert at 80 percent or above	date functions, fetch with criteria, sum(), if-else, Map, List, invokeurl

Code 5.1: DS-01 – Expense amount validation

```
// DS-01 | Form: Expenses | Event: On Add / On Edit ->
On Validate
// Purpose: Reject an expense whose amount is not
greater than zero
if(input.Amount == null || input.Amount <= 0)
{
    alert "Amount must be greater than zero.";
    cancel submit;
}
```

Code 5.2: DS-02 – Budget amount validation

```
// DS-02 | Form: Monthly_Budget | Event: On Add / On
Edit -> On Validate
// Purpose: Reject a budget whose amount is not greater
than zero
if(input.Budget_Amount == null || input.Budget_Amount
<= 0)
{
    alert "Budget amount must be greater than zero.";
    cancel submit;
}
```

Code 5.3: DS-03 – Budget comparison and e-mail alert

```
// DS-03 | Form: Expenses | Event: On Add / On Edit -> On
Success
// Purpose: Alert the student at 80% of the monthly
budget or when it is exceeded
monthStart = input.Expense_Date.toStartOfMonth();
monthEnd = input.Expense_Date.toEndOfMonth();
spent = Expenses[Added_User == zoho.loginuser &&
Expense_Date >= monthStart
                  && Expense_Date <=
monthEnd].sum(Amount);
budgetRec = Monthly_Budget[Added_User == zoho.loginuser
&& Budget_Month >= monthStart
                           && Budget_Month <= monthEnd];
if(budgetRec.count() > 0)
{
    limit = budgetRec.Budget_Amount;
    msg = "";
    if(spent > limit)
    {
        msg = "You have exceeded your monthly budget.
Spent: Rs. " + spent + " of Rs. " + limit;
    }
    else if(spent >= limit * 0.8)
    {
        msg = "You have used 80% or more of your monthly
budget. Spent: Rs. " + spent + " of Rs. " + limit;
    }
    if(msg != "")
    {
        recipient = Map();
        recipient.put("email", zoho.loginuserid);
        toList = List();
        toList.add(recipient);
        sender = Map();
        sender.put("name", "SPEND  SMART");
        sender.put("email", zoho.adminuserid);
        mail = Map();
```

```
        mail.put("sender", sender);
        mail.put("to", toList);
        mail.put("subject", "SPEND  SMART budget alert");
        mail.put("htmlContent", "<p>" + msg + "</p>");
        resp = invokeurl
        [
            url :"https://api.brevo.com/v3/smtp/email"
            type :POST
            parameters:mail.toString()
            headers:{"Content-Type":"application/json"}
            connection:"brevo"
        ];
        info resp;
    }
}
```

DS-01 and DS-02 run when an expense or a budget is added or edited. Each checks that the amount is greater than zero, displays an alert and cancels the submission otherwise, so an invalid record is never stored. They implement the validation required by FR-07.

5.6 Workflows and Automation

Table 5.3: Workflows Implemented

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Expense Validation	Form Workflow	On Add / On Edit → On Validate of Expenses	Always	Cancel submit with an alert if the amount is not positive	DS-01
Budget Validation	Form Workflow	On Add / On Edit → On Validate of Monthly Budget	Always	Cancel submit with an alert if the amount is not positive	DS-02
Budget Alert	Form Workflow	On Add / On Edit → On Success of Expenses	Spending ≥ 80% of the budget or above it	Send e-mail alert through Brevo	DS-03
Other workflow types	Schedule / Approval / Blueprint / Batch	Not applicable	Not applicable	Not used; see Section 8.5	Not applicable

All three workflows are form workflows, because the automation is triggered by saving a record. The budget comparison is performed when an expense is saved rather than on a timer. A scheduled workflow, an approval process and a Blueprint were not part of this version.

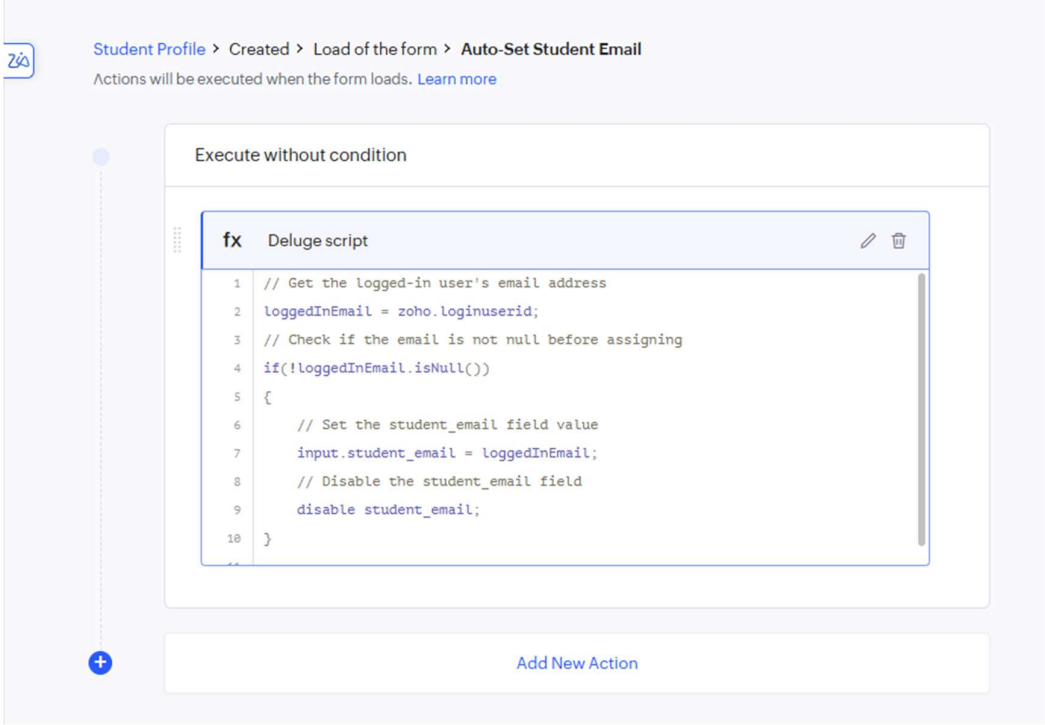


Figure 5.6: Form Workflow Configuration

5.7 Integrations

Table 5.4: Integrations Implemented

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
Brevo	Contacts module; transactional e-mail API [8]	Deluge invokeurl in DS-03 using a Zoho Creator connection	Connection with the Brevo API key	Alert message and the student's e-mail address sent to Brevo

The only integration is Brevo, which delivers the budget alert e-mails to students. The connection is configured once in Zoho Creator, and DS-03 calls the Brevo API through it when the alert condition is met. No webhook, Zoho service integration or Postman test is part of this version.

5.8 User Management and Role-Based Access

One role, Student, was configured to implement the permission matrix of Table 4.4. A student can add, edit, delete and view records in the two forms and can open the dashboard and the six reports. The Student (Group Context) role is a future feature and was not configured.

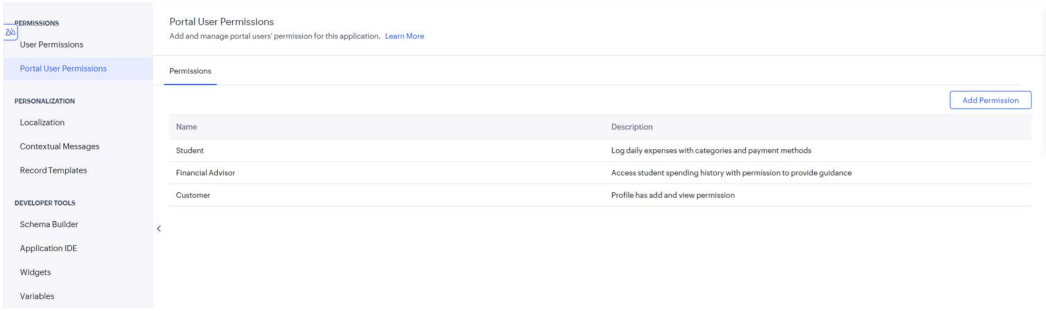


Figure 5.8: Roles and Permission Profiles Configuration

5.9 AI and Analytics Features

No Zia or other AI feature is used in the application. The Deluge scripts were written and tested manually, and the application does not need prediction or text analysis.

5.10 Pro-Code Extension

A pro-code extension was not used. Forms, validation, reports, charts, the dashboard, the alert logic and access control were achieved with configuration and short Deluge scripts, so Zoho Catalyst, JavaScript widgets and custom APIs were not necessary.

5.11 Mobile and Portal Access

The application is a web application and can be opened in a mobile browser or in the Zoho Creator mobile app, so a student can record an expense from a phone soon after spending the money. The mobile layout and features depend on the application configuration and on the edition of Zoho Creator. Mobile use was not covered by a dedicated test case.

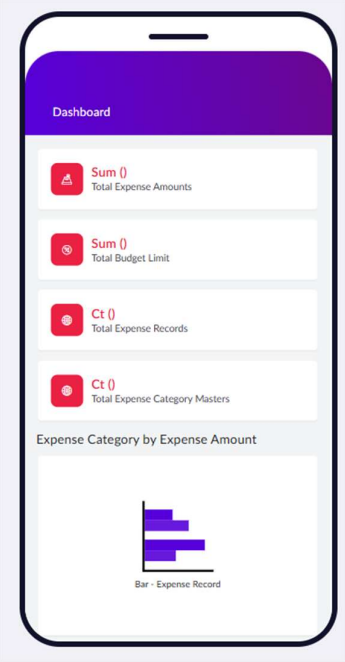


Figure 5.9: SPEND SMART on a Mobile Device

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing was planned at four levels. Unit testing checks each form on its own: mandatory fields, drop-down choices, the auto-generated Expense ID and the positive-amount rule. Script testing checks DS-01, DS-02 and DS-03 with valid and invalid inputs, including spending below, at and above 80 percent of the budget. Integration testing checks the reports, the dashboard and the Brevo e-mail. Access testing checks that a student cannot see the records of another student.

6.2 Test Cases

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-01, FR-02	Add an expense with valid data	Open Add Expense; fill all mandatory fields; submit	Record saved with an auto-generated Expense ID	Record saved with auto-generated Expense ID (EXP-); shown in All Expenses report	Pass
TC-02	FR-02	Submit an expense with a mandatory field empty	Leave Amount empty; submit	Validation error; record not saved	Validation message shown for the empty mandatory field; record not saved	Pass
TC-03	FR-07	Enter a zero or negative amount (DS-01)	Enter 0 as the amount; submit	Alert shown; submission cancelled	Alert 'Amount must be greater than zero.' shown; submission cancelled	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-04	FR-05	Edit an expense	Change the amount of an existing expense; update	Updated record saved	Edited amount saved; reports and totals show the new value	Pass
TC-05	FR-05	Delete an expense	Delete an existing expense	Record removed from reports and totals	Record deleted; removed from reports and totals	Pass
TC-06	FR-06, FR-08	Set a monthly budget	Open Monthly Budget; select month; enter amount and note; submit	Budget saved for the month	Budget saved for the selected month with the note	Pass
TC-07	FR-07	Enter a zero or negative budget (DS-02)	Enter 0 as the budget amount; submit	Alert shown; submission cancelled	Alert 'Budget amount must be greater than zero.' shown; submission cancelled	Pass
TC-08	FR-03, FR-04	Check drop-downs and currency display	Open the Add Expense form; view the amount in a report	Category and payment method offer choices; amounts shown in ₹	Category and Payment Method offered predefined choices; amounts displayed with ₹	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-09	FR-09	View All Expenses	Open All Expenses report	All own expenses listed, sorted by date	All own expenses listed, sorted by date	Pass
TC-10	FR-09	View Today's and This Month's reports	Open both reports	Only expenses of today and of the current month are listed	Today's report listed only today's expenses; This Month's report listed only the current month	Pass
TC-11	FR-10	View Category-wise and Payment Method reports	Open both reports	Records grouped with totals	Records grouped by category and by payment method with totals	Pass
TC-12	FR-11	View the Monthly Trend report	Open Monthly Trend	Spending shown month by month	Monthly Trend showed total spending for each month	Pass
TC-13	FR-12	Search and filter expenses	Search by name; filter by category	Only matching expenses displayed	Only expenses matching the search text and category filter were displayed	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-14	FR-13	Dashboard reflects a new expense	Add an expense; open the dashboard	Cards, charts, recent table and status updated	Summary cards, pie chart, bar chart, recent table and budget status updated	Pass
TC-15	FR-14	Alert at 80 percent of the budget (DS-03)	Save expenses so that the month's total reaches 80% of the budget	E-mail alert received through Brevo	E-mail alert for 80% of the budget received through Brevo	Pass
TC-16	FR-14	Alert when the budget is exceeded (DS-03)	Save an expense that takes the total above the budget	E-mail alert for exceeded budget received	E-mail alert for exceeded budget received through Brevo	Pass
TC-17	FR-15, NFR-03	Privacy between students	Sign in as a second student; open reports and forms	Records of the first student are not visible	Second student could not see the first student's expenses or budgets	Pass

6.3 Debugging Deluge Scripts

Deluge scripts are debugged with three techniques. The info statement prints intermediate values, such as the monthly total and the budget in DS-03 and the response of the Brevo call, to the execution log. The execute option of the Deluge editor and the workflow log show which statement ran and whether an error occurred. The alert messages shown on form submission confirm the cancel-submit behaviour of DS-01 and DS-02.

6.4 Defect Log

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
D-01	A budget of 0 was accepted when an existing budget was edited	DS-02 was attached to On Add only	DS-02 attached to On Add and On Edit	Closed
D-02	Budget alert used the wrong month for an expense dated in a previous month	Criteria used the current date instead of the expense date	DS-03 changed to use the month of Expense_Date	Closed

6.5 User Acceptance Testing

Table 6.3: User Acceptance Feedback

User	Role Tested	Feedback	Action Taken
Student tester 1	Student	The expense form is quick to fill and the dashboard is easy to read	No change required
Student tester 2	Student	Colour-coded budget status and the e-mail alert are useful	Alert message wording retained

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

The application is developed in Zoho Creator and is accessed by students through its application URL. It was built and tested in one Zoho Creator environment and then shared with the students through its application URL.

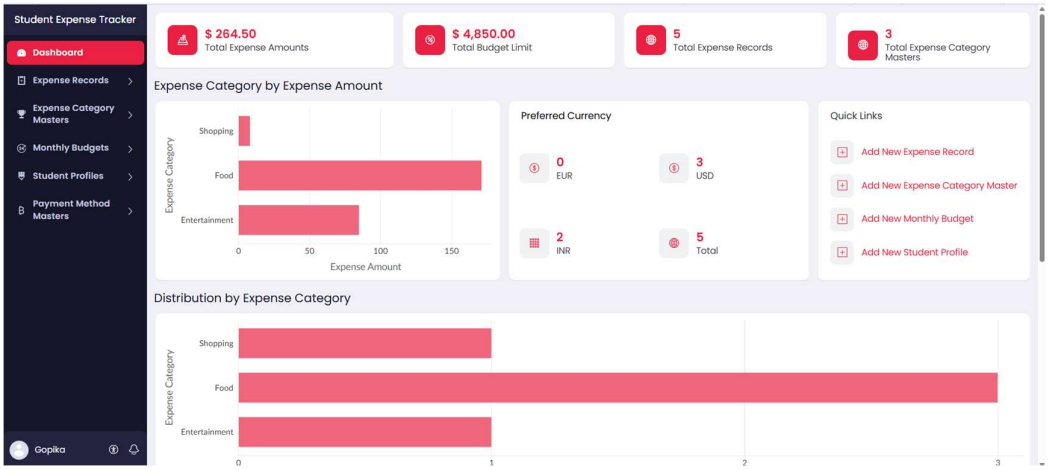


Figure 7.1: Environments / Publish to Production

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done (✓)
1	All test cases executed in Development / Stage	✓
2	Sample/test data removed or isolated	✓
3	Roles, permissions and users configured in Production	✓
4	Connections authorised in Production	✓
5	Schedules and workflows enabled	✓
6	Application published and URL verified	✓
7	Version / release notes recorded	✓

7.2 Performance Optimisation

The application uses two forms, so the main performance decisions concern reports, the dashboard and the alert script. Reports apply the student's record-level criteria and date criteria so that only relevant records are loaded. DS-03 calculates the monthly total with one criteria-based sum instead of looping through the records, and no fetch is placed inside a loop. The effect of each optimisation can be measured and entered in Table 7.2.

Table 7.2: Performance Optimisation Results

Optimisation Applied	Component	Before	After	Improvement
Limit reports to the current student's records and the required period	Today's and This Month's reports	Report loaded the records of all months	Report limited to the signed-in student's current-month records	Fewer records loaded; faster display
Use a single criteria-based sum in DS-03 instead of looping through records	DS-03	DS-03 added the amounts in a loop over expenses	DS-03 uses one criteria-based sum	One query instead of a loop
Show only the needed columns in reports	All Expenses report	Report showed all columns	Report shows only the needed columns	Easier reading on small screens

7.3 Security Features

Security rests on the controls provided by Zoho Creator. Students sign in with a Zoho account, permissions limit what a student can do, record-level criteria separate the data of different students, and validation prevents invalid amounts from being stored. The controls are listed in Table 7.3.

Table 7.3: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
Role and permission profile	Forms, reports and dashboard	Unauthorised creation, change or deletion of records
Record-level criteria (Added User = signed-in user)	All forms and reports	One student seeing another student's financial data
Mandatory fields	Expenses and Monthly Budget forms	Incomplete records
Validation scripts DS-01 and DS-02	Expenses and Monthly Budget forms	Zero or negative amounts

Security Control	Where Applied	Threat Mitigated
Connection for the Brevo API	DS-03	Exposure of the API key inside script code
Audit trail and change history	Application settings	Untraceable changes to records

7.4 Governance and Administration

The application owner is responsible for keeping the application accurate and available: maintaining the category and payment-method choices, reviewing the scripts after any change, and managing the Brevo connection. The audit trail and change history of the platform show who changed a record and when, which supports accountability.

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

The working application is demonstrated with one scenario for a student. Step 1: the student sets a monthly budget in the Monthly Budget form and records an expense in the Add Expense form; DS-01 and DS-02 validate the amounts. Step 2: as more expenses are recorded, the dashboard updates its cards, charts and budget status, and when spending reaches 80 percent of the budget an e-mail alert is sent through Brevo. Step 3: the student opens the category-wise, payment-method and monthly-trend reports to see where the money was spent.

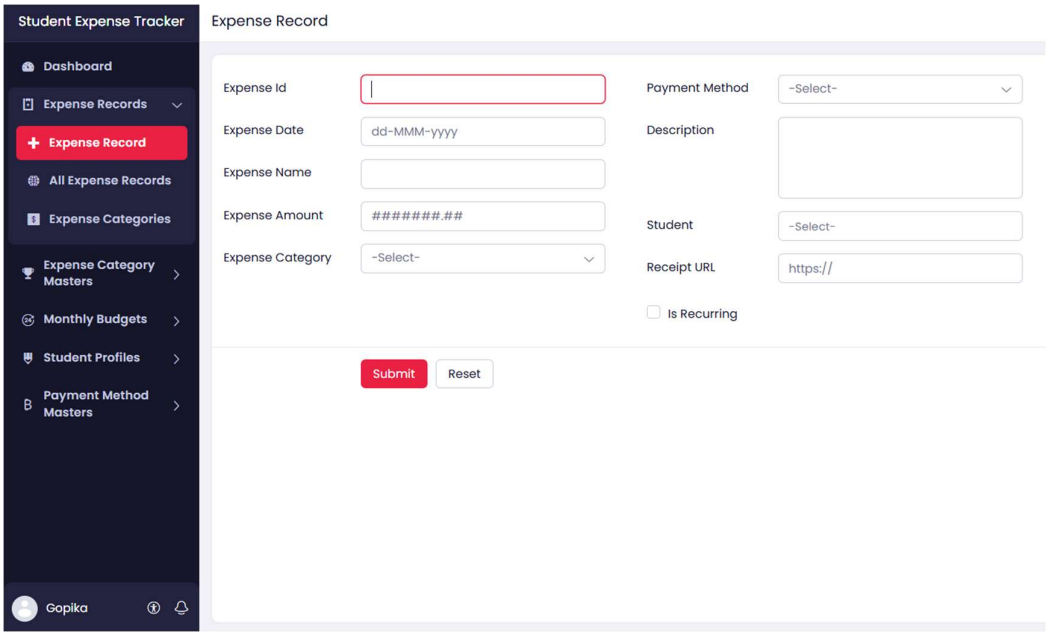


Figure 8.1: Scenario Step 1 – Student sets a budget and records an expense

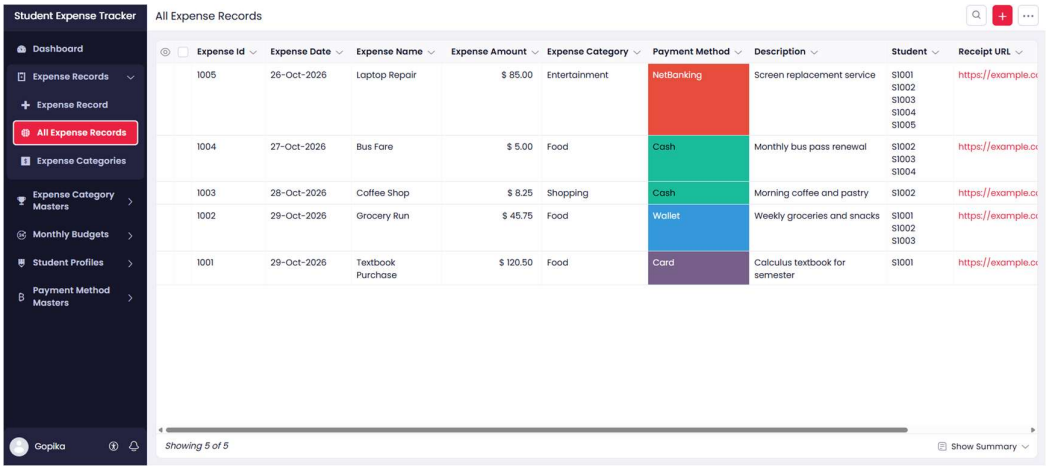


Figure 8.2: Scenario Step 2 – Dashboard updates and the budget alert is sent

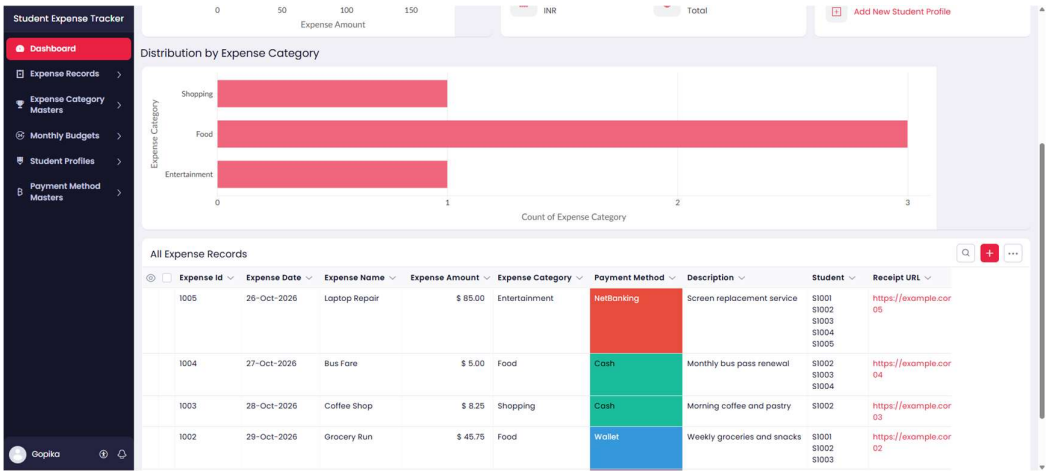


Figure 8.3: Scenario Step 3 – Student reviews the analysis reports

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
To design forms and fields for expenses and budgets	Sections 4.2, 4.3, 5.2	Two forms with validation and an auto-generated Expense ID (Figures 4.2, 5.2–5.3)
To implement validation and budget comparison with Deluge	Sections 5.5, 5.6	Scripts DS-01 to DS-03 and three form workflows (Code 5.1–5.3)
To provide reports and a dashboard	Sections 5.3, 5.4	Six reports and the dashboard with charts (Figures 5.4, 5.5, 5.9)
To send e-mail alerts through Brevo	Section 5.7	DS-03 and the Brevo connection (Code 5.3, Figure 5.7)
To keep each student's data private	Sections 4.6, 5.8	Student role and record-level criteria (Table 4.4, Figure 5.8)
To evaluate through test cases and feedback	Chapter 6	Seventeen test cases (Table 6.1)

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Recording an expense	Writing in a notebook or note, often without category or payment method	Structured form with mandatory fields and drop-downs	Consistent, complete records
Totals for today and the month	Added by hand	Today's and This Month's reports and dashboard cards	Automatic totals
Spending by category and payment method	Prepared manually if at all	Category-wise and Payment Method reports and pie chart	Habits visible at a glance
Budget comparison	Estimated by the student	Calculated automatically with remaining or exceeded amount	Timely awareness of the budget status
Warning before overspending	None	E-mail alert at 80 percent and when exceeded	Early warning
Privacy of data	Depends on where the notes are kept	Record-level criteria for each student	Private, controlled access

The comparison is qualitative. Quantitative values such as the time saved per entry were not measured and should be added only if they are measured.

8.4 Limitations of the Low-Code Approach

The constraints met on Zoho Creator were mostly related to the boundaries of the platform. The options of pages and charts are limited to the components of the UI Builder, so a fully custom design would need a pro-code widget. Deluge has limits on statements and API calls that are not reached in this small application. Some features depend on the edition and plan, such as the number of users, the availability of environments and the integrations.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

On the spectrum from no-code to pro-code, most of SPEND SMART is no-code configuration: the two forms, field types, six reports, the dashboard with its charts and the permissions were created by configuration in the builder. Low-code Deluge was needed for the three scripts that validate amounts and compare spending with the budget, and for the call to the Brevo API.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

SPEND SMART was developed on Zoho Creator to help college students record expenses, manage monthly budgets and understand their spending. Two forms store the data, Deluge scripts validate amounts and compare spending with the budget, six reports and a dashboard summarise the data, an integration with Brevo sends alerts at 80 percent of the budget and when it is exceeded, and record-level criteria keep every student's data private.

9.2 Future Enhancements

- Group expense sharing, so that students can view shared expenses and track shared budgets (the Student – Group Context role).
- Budget limits for each category in addition to the monthly budget.
- A scheduled workflow that sends a weekly or monthly spending summary.
- Analytics with Zoho Analytics or Zia on spending habits and forecasts.
- Export of reports to PDF or Excel for sharing.
- Push notifications in the Zoho Creator mobile app.
- Import of expenses from bank statements.

9.3 Learning Outcomes and Career Relevance

Developing this application gave practical experience of the complete low-code life cycle: analysing a real problem, writing requirements, modelling data, building forms and reports, writing Deluge scripts, integrating an external service, configuring access, and testing. Designing the budget comparison taught how a small amount of code can enforce business rules over related data.

REFERENCES

[1] Zoho Corporation, “Zoho Creator Help Documentation.” [Online]. Available: https://www.zoho.com/creator/help/ (Accessed: October 2026).

[2] Zoho Corporation, “Deluge Script Reference Guide.” [Online]. Available: https://www.zoho.com/deluge/ (Accessed: October 2026).

[3] R. Sanchis, Ó. García-Perales, F. Fraile, and R. Poler, “Low-code as enabler of digital transformation in manufacturing industry,” *Applied Sciences*, vol. 10, no. 1, Art. no. 12, 2020, doi: 10.3390/app10010012.

[4] A. C. Bock and U. Frank, “Low-code platform,” *Business & Information Systems Engineering*, vol. 63, no. 6, pp. 733–740, 2021, doi: 10.1007/s12599-021-00726-8.

[5] D. Di Ruscio, D. Kolovos, J. de Lara, A. Pierantonio, M. Tisi, and M. Wimmer, “Low-code development and model-driven engineering: Two sides of the same coin?,” *Software and Systems Modeling*, vol. 21, no. 2, pp. 437–446, 2022.

[6] A. Lusardi and O. S. Mitchell, “The economic importance of financial literacy: Theory and evidence,” *Journal of Economic Literature*, vol. 52, no. 1, pp. 5–44, 2014.

[7] R. H. Thaler, “Mental accounting matters,” *Journal of Behavioral Decision Making*, vol. 12, no. 3, pp. 183–206, 1999.

[8] Brevo, “Brevo API Documentation.” [Online]. Available: https://developers.brevo.com/ (Accessed: October 2026).

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	Y	SPEND SMART; Dashboard, Expenses, Budget and Analysis menu groups	5.1	Fig. 5.1
II	Forms with ≥ 8 distinct field types	N	Six field types (below the eight required)	5.2	Tables 4.2–4.3
II	Lookup field (relationship between forms)	N	No lookup; budget and expenses are matched by month	4.2, 5.2	Fig. 4.2
II	Subform (parent–child data)	N	Not used (no parent–child data)	4.2, 5.2	Not applicable
II	Field validations / layout rules	Y	Required fields; positive-amount validation DS-01, DS-02	5.2	Tables 4.2–4.3; Code 5.1–5.2
II	List report with filter, grouping, custom action	N	Grouped list reports with search and filter; no custom action	5.3	Fig. 5.4
II	Kanban / Calendar / Timeline / Map report	N	Not used	5.3	Not applicable
II	Pivot table / chart report	Y	Monthly Trend chart report	5.3	Table 5.1
II	Page / dashboard built with UI Builder	Y	SPEND SMART Dashboard	5.4	Fig. 5.5

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
II	Deluge – form events (validate / success / field actions)	Y	DS-01, DS-02 (On Validate); DS-03 (On Success)	5.5	Code 5.1–5.3
II	Deluge – functions, fetch, loops, collections	Y	DS-03: criteria fetch, sum(), Map and List collections	5.5	Code 5.3
II	Email / in-app notifications	Y	DS-03 budget alert e-mail through Brevo	5.5, 5.6	Code 5.3
II	Form workflow	Y	Three form workflows	5.6	Fig. 5.6; Table 5.3
II	Scheduled workflow	N	Not used	5.6	Not applicable
II	Approval process	N	Not used	5.6	Not applicable
II	Blueprint	N	Not used	5.6	Not applicable
II	Batch / report workflow	N	Not used	5.6	Not applicable
II	Testing & debugging evidence	Y	Seventeen test cases with results; Deluge debugging	6	Table 6.1; Fig. 6.1
III	Integration with a Zoho service	N	Brevo is not a Zoho service	5.7	Not applicable
III	Connection / REST API call (invokeurl)	Y	invokeurl call to the Brevo API using a connection	5.7	Code 5.3; Fig. 5.7
III	Webhook or Creator REST API	N	Not used	5.7	Not applicable
III	Users, roles and permission profiles	Y	Role: Student	5.8	Fig. 5.8; Table 4.4

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
III	Record-level access criteria / customer portal	Y	Record-level criteria: Added User = signed-in user	5.8	Table 5.1
III	Zia / AI feature	N	Not used	5.9	Not applicable
III	Analytics (charts / Zoho Analytics)	Y	Dashboard charts and summary cards	5.9	Section 5.9
IV	Performance optimisation applied & measured	N	Applied; qualitative before / after comparison	7.2	Table 7.2
IV	Deployment across environments	N	Single Zoho Creator environment	7.1	Fig. 7.1
IV	Security controls (encryption, OAuth, permissions)	Y	Permissions, record criteria, validation	7.3	Table 7.3
IV	Governance (audit trail, administration)	Y	Audit trail and administration settings	7.4	Fig. 7.2
V	Pro-code extension (Catalyst / widgets / custom functions)	N	Not required	5.10	Not applicable
V	Low-code limitations & spectrum analysis	Y	Limitations and spectrum analysis	8.4, 8.5	Sections 8.4, 8.5
Not applicable	Mobile app / portal access	N	Platform capability; opened in a mobile browser	5.11	Fig. 5.9

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2		
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6		
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7–5.9		
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7		
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3		

APPENDIX C – COMPLETE DELUGE SOURCE CODE

C.1 Expenses – DS-01 Expense amount validation

```
// DS-01 | Form: Expenses | Event: On Add / On Edit ->
On Validate
// Purpose: Reject an expense whose amount is not
greater than zero
if(input.Amount == null || input.Amount <= 0)
{
    alert "Amount must be greater than zero.";
    cancel submit;
}
```

C.2 Monthly_Budget – DS-02 Budget amount validation

```
// DS-02 | Form: Monthly_Budget | Event: On Add / On
Edit -> On Validate
// Purpose: Reject a budget whose amount is not greater
than zero
if(input.Budget_Amount == null || input.Budget_Amount
<= 0)
{
    alert "Budget amount must be greater than zero.";
    cancel submit;
}
```

C.3 Expenses – DS-03 Budget comparison and e-mail alert

```
// DS-03 | Form: Expenses | Event: On Add / On Edit -> On
Success
// Purpose: Alert the student at 80% of the monthly
budget or when it is exceeded
monthStart = input.Expense_Date.toStartOfMonth();
monthEnd = input.Expense_Date.toEndOfMonth();
spent = Expenses[Added_User == zoho.loginuser &&
Expense_Date >= monthStart
                  && Expense_Date <=
monthEnd].sum(Amount);
budgetRec = Monthly_Budget[Added_User == zoho.loginuser
&& Budget_Month >= monthStart
                           && Budget_Month <= monthEnd];
if(budgetRec.count() > 0)
{
    limit = budgetRec.Budget_Amount;
    msg = "";
    if(spent > limit)
    {
        msg = "You have exceeded your monthly budget.
Spent: Rs. " + spent + " of Rs. " + limit;
    }
    else if(spent >= limit * 0.8)
```

```
    {
        msg = "You have used 80% or more of your monthly
budget. Spent: Rs. " + spent + " of Rs. " + limit;
    }
    if(msg != "")
    {
        recipient = Map();
        recipient.put("email", zoho.loginuserid);
        toList = List();
        toList.add(recipient);
        sender = Map();
        sender.put("name", "SPEND  SMART");
        sender.put("email", zoho.adminuserid);
        mail = Map();
        mail.put("sender", sender);
        mail.put("to", toList);
        mail.put("subject", "SPEND  SMART budget alert");
        mail.put("htmlContent", "<p>" + msg + "</p>");
        resp = invokeurl
        [
            url :"https://api.brevo.com/v3/smtp/email"
            type :POST
            parameters:mail.toString()
            headers:{"Content-Type":"application/json"}
            connection:"brevo"
        ];
        info resp;
    }
}
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

Open the application URL of SPEND SMART in a web browser (or in the Zoho Creator mobile app). Sign in with your Zoho account. After signing in, the dashboard is displayed, and the menu shows the Expenses, Budget and Analysis groups. You can see and change only your own records.

D.2 Instructions for Recording Expenses and Budgets

- Open the Monthly Budget form, select the month and enter your budget amount and an optional note.
- Open the Add Expense form and enter the expense date, name, category, amount, payment method and an optional description.
- If an alert appears, correct the entry; amounts must be greater than zero and required fields must be filled.
- Open All Expenses to search, filter, edit or delete an entry.
- Read the e-mail alert that is sent when your spending reaches 80 percent of the budget or exceeds it.

D.3 Instructions for Viewing Reports and the Dashboard

- Open the dashboard to see total, today's and month-to-date spending, the budget and the remaining amount.
- Read the category pie chart and the monthly bar chart to see where and when money was spent.
- Check the colour-coded budget status to see whether you are under, at or over budget.
- Open Today's Expenses and This Month's Expenses for recent spending.
- Open the Category-wise, Payment Method and Monthly Trend reports for summaries.

APPENDIX E – EVALUATION RUBRIC

Student Name	GOPIKA V	Register No.	241501058
Project Title	SPEND SMART	Date	

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		**100**	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner

Name: _____

Signature of External Examiner

Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

Student Name	GOPIKA V	Register No.	241501058
Department / Section	Artificial Intelligence and Machine Learning	Date of Viva	October 2026
Application URL Verified	Yes / No	Demo Duration	_____ min

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	**100**	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Name: _____ Name: _____